ON THIS PAGE

- Introduction
- Goals
- Hardware & Software Needed +
- Controlling over Wi-Fi +
- Programming the Board
- Testing It Out +
- Conclusion

This tutorial refers to a product that has reached its end-of-life status.

Web Server AP Mode with Arduino Nano RP2040 Connect

Learn how to set up your board as an access point, allowing other clients to connect via browser, to control and monitor data.

Author · Karl Söderby Last revision · 07/17/2024

Introduction

The Nano RP2040 Connect features a Wi-Fi module and an RGB LED among many other things. In this tutorial we will take a look at how we turn our board into an access point, and control the built-in RGB through a browser connected to the access point.

> *Note: if you need help setting up your environment to use your Arduino Nano RP2040 board, please refer to this installation guide.*

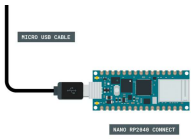
The goals of this project are:

- Create an access point (AP).
- Set up a web server.
- Connect to the server through a browser.
- Control the RGB LED.

Hardware & Software Needed

- Arduino IDE (**online** or **offline**).
- **WiFiNINA** library.
- **Arduino Nano RP2040 Connect**.

Circuit



Plug in the Nano RP2040 Connect to your computer.

Controlling over Wi-Fi

There are multiple ways we can access our board over Wi-Fi. In this tutorial, we will turn our board into a web server, that will listen for incoming GET requests.

Simply explained, we will set up our board as an access point, and start hosting a web server on a

now show up in the list of available Wi-Fi networks on for example your smartphone or computer.

If we connect to this access point, and enter the board's address in the browser of a computer/browser on the same network, we make a request to this server. The server responds with a set of HTML instructions, which can then be viewed in the browser.

Controlling an Arduino Through the Browser

Inside the HTML instructions that we print, or send to the browser, we can create **buttons.** These buttons can be modified to add something to the URL. Here is an example of how a button is created:

```
1  <button class='red' typ
```

The most important is what we put inside `href`, which in this case is `/RH`. Whenever this button is clicked, it will update the URL of the page to `http://<board-ip>/RH`. After this happens, the program will go through a set of conditionals, that checks whether this button has been pressed:

```
if (currentLine.endsWit
          doSomething()
        }
```

By using this method, we can set up many more buttons that can control different aspects of our Arduino, and is a great building block for creating a control interface for your board that can be controlled through a browser, over Wi-Fi.

Programming the Board

We will now get to the programming part of this tutorial.

1. First, let's make sure we have the drivers installed. If we are using the Cloud Editor, we do not need to install anything. If we are using an offline editor, we need to install it manually. This can be done by navigating to **Tools > Board > Board Manager...**. Here we need to look for the **Arduino Mbed OS Nano Boards** and install it.
2. Now, we need to install the libraries needed. If we are using the Cloud Editor, there is no need to install anything. If we

Tools > Manage libraries.., and search for **WiFiNINA** and install it.

3 We can now take a look at some of the core functions of this sketch:

`char ssid[] = ""` - create a network name.

`char pass[] = ""` - create a network password.

`WiFi.beginAP(ssid, pass)` - creates an access point with the stored credentials.

`WiFiServer server(80)` - creates a server that listens for incoming connections on the specified port.

`WiFiClient client` - creates a client that can connect to to a specified internet IP address.

`server.begin()` - tells the server to begin listening for incoming connections.

`client.connected` - checks for connected clients.

`client.available` - checks for available data.

`client.read` - reads the available data.

`client.print()` - print something to the client (e.g. html code).

`client.stop()` - closes the connection.

The sketch can be found in the snippet below. Upload the sketch to the board.

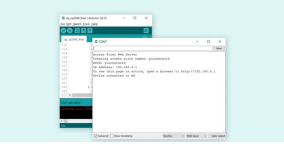
Note: both `ssid` and `pass` needs to be a minimum of 8 characters

```
#include <SPI.h>
#include <WiFiNINA.h

char ssid[] = "yourn
char pass[] = "yourp
int keyIndex = 0;

int status = WL_IDLE
WiFiServer server(80

void setup() {
  //Initialize seria
  Serial.begin(9600)
  while (!Serial) {
    ; // wait for se
  }

  Serial.println("Ac

  pinMode(LEDR, OUTP
  pinMode(LEDG, OUTP
  pinMode(LEDB, OUTP

  // check for the W
  if (WiFi.status()
    Serial.println("
    // don't continu
    while (true);
```

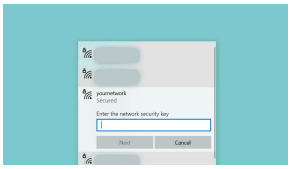
Testing It Out

After the code has been successfully uploaded to the board, we need to open the Serial Monitor to initialize the program.

When we open it, it will attempt to create an access point, and if it is successful, it will print out the boards IP address in the Serial Monitor.

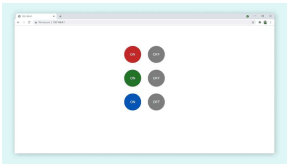


We will now need to connect to this access point through either our smartphone or computer, by browsing the list of available networks.



Finding your network.

Once we have connected, we need to copy the enter the address that was printed in the Serial Monitor, in the browser of our device. We should now see a web page with a set of colored buttons.



Accessing the board from the browser.

We can now interact with the different buttons. The buttons available are used to control the built-in RGB LED on the Nano RP2040 Connect. There's six buttons in total, three to turn ON the different colors, and three to turn them off.

Troubleshoot

If the code is not working, there are some common issues we can troubleshoot:

- We have entered the wrong

We have not installed the **WiFiNINA** library.

The WiFiNINA library is updated (can be done in the library manager).

Conclusion

In this tutorial, we have turned our Nano RP2040 Connect into a web server, which different clients can connect to. We have then used the /GET method in order to change the RGB LED on the board, through a set of buttons accessible from the browser.

This method can be quite useful for turning something ON or OFF remotely, and can be an ideal solution for building smart home applications. You can easily replace the control of the RGB LED with something else, such as controlling a motor, relays or other actuators.

Was this article helpful?